```
<!-- type: epic -->
```

#0074 — Onboarding pipeline (workflow-driven, prospect → trial group → academy team)

> Replaces the kanban-style draft. Same real-world journey, different architecture: the onboarding sequence is expressed as a **chain of workflow tasks** in the existing Workflow module, with `tt_prospects` as the entity that carries identity across stages and a new persona-dashboard widget as the visualisation surface. Half the new code, sits inside an architecture the team already understands.

Problem

Three distinct gaps in today's TalentTrack:

1. The front half of the recruitment journey has no infrastructure. A scout sees a player at a match, gets parent contact details, the academy invites the player to a single test training, the test training happens, the HoD decides. None of this lives in the system today. Clubs run it on phone calls and shared spreadsheets, then re-enter the player into TalentTrack at the trial-case stage with all earlier context lost. The existing `Trials` module starts at the trial group, not before it.

2. The back half assumes a fixed-period trial. `tt_trial_cases` has `start_date / end_date / decision`. The real-world trial group runs as a rolling weekly training that may continue for an open-ended period — a player can be in the trial group for a full season (alongside their own club) before any team-offer conversation. Today clubs either set very long end dates or chain cases. The existing decision enum (`admit / deny_final / deny_encouragement`) doesn't represent "offered a team position, declined, stays in trial group."

3. Operators running the funnel have no visualisation surface. Clubs running 30+ prospects across the funnel want to see at a glance "we have 12 prospects logged, 4 invited, 3 at test-training stage, 8 in trial group, 2 awaiting team-offer decision." Today the closest surface is the Trials admin list, which is one flat table at decision time and shows nothing earlier.

The architectural insight that shapes the rest of this spec: **the journey is a chain of tasks, not a custom kanban**. A scout completes a "log prospect" task; that completion spawns an "invite to test training" task assigned to the HoD; completing that spawns a "confirm with parent" task; and so on. The Workflow module already has every primitive needed — `TaskEngine`, `TaskTemplate`, `TaskStatus` (open / in_progress / completed / overdue / skipped / cancelled), `ChainStep` for declarative spawn-on-complete, `TaskContext` for propagating identity, even an HoD-tier overview at `?tt_view=tasks-dashboard`. The work is to add prospect-stage data, the templates that wire the chain, and a dashboard widget that pivots existing task data by stage. No parallel state machine; no bespoke kanban; no custom "drag a card" affordance — completing a task spawns the next, and that *is* the advancement.

A fourth issue surfaces when GDPR is taken seriously: a prospect's data is personal data collected before any contractual relationship. The legal basis is consent. Active prospects retain their data through their chain; stale prospects (logged but never progressed, or terminal-state for some retention period) must be auto-purged. The new `tt_prospects` entity is sized for this lifecycle.

Proposal

A four-child epic. The shape:

Shape — four child specs

- **`feat-prospects-entity`** — the missing data layer: `tt_prospects` (identity, contact, scouting context) and `tt_test_trainings` (the scheduled session). Lifecycle is driven by the workflow tasks, not by a status column on the prospect — the prospect's "current stage" is derived from their most recent task. Includes the GDPR auto-purge cron for stale prospects.
- **`feat-onboarding-task-templates`** — five new `TaskTemplate` classes wired into the Workflow module. The templates declare their own `chainSteps()` so the chain wires itself end-to-end — completing the test-training-outcome task with decision `admit_to_trial` automatically creates a `tt_trial_cases` row and (separately) the next workflow task. No custom orchestration code; the engine handles it.
- **`feat-trial-cases-rolling-membership`** — small but important rework of the existing `Trials` module. Three new decision values (`offered_team_position` , `declined_offered_position` , `continue_in_trial_group`), a `continued_until` column for rolling membership, and a per-club pseudo-team for the trial group so its weekly attendance flows through the existing `tt_attendance` infrastructure (and shows up in the HoD landing's team grid for free).
- **`feat-pipeline-widget`** — a new persona-dashboard widget `OnboardingPipelineWidget` that visualises the chain as columns. Reads from `tt_workflow_tasks` joined to `tt_prospects` and `tt_trial_cases` ; groups by stage; counts active and stale items per stage; renders as a horizontal flow diagram. **This is the primary user-facing surface for the pipeline** — sized to live on the HoD landing as a featured widget, scoped down for scout and academy admin per their respective roles. The widget is the operational pulse, not just a decorative tile.

The four are sequenced. Entity first (everything depends on it). Templates second (gives clubs the everyday flow). Trial-cases rework third (changes existing behaviour, lands on stable foundation). Widget fourth (pivots the data assembled by the first three).

This shape replaces a fully-bespoke kanban surface with a *widget that pivots existing data*. The savings are real: ~1,200 LOC instead of ~2,200, and every operator who already understands the persona dashboard understands the pipeline view by analogy.

Scope

1. `feat-prospects-entity`

Two new tables, both deliberately small. Status is *not* a column on the prospect — the workflow tasks are the source of truth for state.

`tt_prospects` — identity and context that persist across the chain.

```
CREATE TABLE {prefix}tt_prospects (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    club_id BIGINT UNSIGNED NOT NULL,

    -- identity (minimum to follow up)
    first_name VARCHAR(255) NOT NULL,
```

```
    last_name VARCHAR(255) NOT NULL,
    date_of_birth DATE DEFAULT NULL,
    age_group_lookup_id BIGINT UNSIGNED DEFAULT NULL,

    -- discovery context
    discovered_at DATE NOT NULL,
    discovered_by_user_id BIGINT UNSIGNED NOT NULL,
    discovered_at_event VARCHAR(255) DEFAULT NULL,
    current_club VARCHAR(255) DEFAULT NULL,
    preferred_position_lookup_id BIGINT UNSIGNED DEFAULT NULL,
    scouting_notes TEXT DEFAULT NULL,

    -- contact (consent-captured at scout time)
    parent_name VARCHAR(255) DEFAULT NULL,
    parent_email VARCHAR(255) DEFAULT NULL,
    parent_phone VARCHAR(50) DEFAULT NULL,
    consent_given_at DATETIME DEFAULT NULL,

    -- transitions out
    promoted_to_player_id BIGINT UNSIGNED DEFAULT NULL,
    promoted_to_trial_case_id BIGINT UNSIGNED DEFAULT NULL,

    -- lifecycle (the prospect row itself; status of the JOURNEY lives in tt_workflow_tasks)
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    archived_at DATETIME DEFAULT NULL,
    archived_by BIGINT UNSIGNED DEFAULT NULL,
    archive_reason VARCHAR(40) DEFAULT NULL,

    PRIMARY KEY (id),
    KEY idx_club (club_id),
    KEY idx_discovered_by (discovered_by_user_id, discovered_at),
    KEY idx_age_group (age_group_lookup_id),
    KEY idx_player (promoted_to_player_id),
    KEY idx_trial (promoted_to_trial_case_id)
);
```

The crucial omission from yesterday's draft: **no `status` column**. A prospect's current stage is the most recent task on its chain. Querying "which prospects are at the test-training stage" becomes "which prospects have an open `record_test_training_outcome` task or a recently-completed `confirm_test_training` task." This avoids the dual-state-machine problem (status on the entity vs status on the task) that bites every pipeline implementation eventually.

`tt_test_trainings` — the scheduled session. Many-to-many to prospects through workflow tasks, not a join table — each prospect gets a `confirm_test_training` task whose `TaskContext` carries `test_training_id`.

```
CREATE TABLE {prefix}tt_test_trainings (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    club_id BIGINT UNSIGNED NOT NULL,
    date DATETIME NOT NULL,
    location VARCHAR(255) DEFAULT NULL,
    age_group_lookup_id BIGINT UNSIGNED DEFAULT NULL,
    coach_user_id BIGINT UNSIGNED NOT NULL,
    notes TEXT DEFAULT NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    created_by BIGINT UNSIGNED NOT NULL,
```

```
    archived_at DATETIME DEFAULT NULL,
    PRIMARY KEY (id),
    KEY idx_club_date (club_id, date),
    KEY idx_age_group (age_group_lookup_id)
);
```

Migration `0059_prospects` creates both tables.

GDPR auto-purge. A daily cron `tt_prospects_retention_cron` finds prospects matching either:

- No active or recently-completed task (last task completed > 30 days ago, or no task ever) AND created_at > 90 days ago, OR
- Most recent task completed with a terminal-decline outcome AND > 30 days since that completion.

For each match: hard-delete the prospect row, hard-delete the linked workflow tasks (cascade by `context.prospect_id`), write a single audit row to `tt_authorization_changelog` with `change_type = 'gdpr_prospect_retention_purge'` and a hash. Retention windows are configurable via two new lookup values (`prospect_retention_days_no_progress` default 90, `prospect_retention_days_terminal` default 30).

Active-chain prospects are never auto-purged. On promotion to player, the prospect row stays as historical record and inherits the player's longer retention via `PlayerDataMap` from #0073.

Matrix entities. Two new entries seeded:

Entity	Player	Parent	Asst Coach	Head Coach	Team Mgr	Scout	HoD	Academy Admin
`prospects`	—	—	—	R team	—	RCD self	RCD global	RCD global
`test_trainings`	—	—	—	R team	—	R global	RCD global	RCD global

Scout sees only their own prospects (`RCD self`). HoD sees everything globally. Per-cell rationale: prospect data is the most consent-sensitive PII in the system; a permissive default leaks it when a scout leaves. Clubs that want broader scout visibility can edit the matrix.

`LegacyCapMapper` bridges new caps `tt_view_prospects`, `tt_edit_prospects`, `tt_view_test_trainings`, `tt_edit_test_trainings`.

`PlayerDataMap` (#0073) registers `tt_prospects` and `tt_test_trainings` as PII tables — on a future GDPR erasure of a prospect-turned-player, the prospect row is part of the erasure manifest.

2. `feat-onboarding-task-templates`

Five `TaskTemplate` classes that express the chain. Each is a small file (~80–120 lines) following the conventions established by the existing five templates (`PostGameEvaluationTemplate`, `PlayerSelfEvaluationTemplate`, etc.). The chain wires itself via each template's `chainSteps()` method — completing one task automatically dispatches the next.

`LogProspectTemplate`. Initiated by a scout via a "+ New prospect" entry-point button (persona dashboard, pipeline widget, or the prospects list). The form is the scout's quick-capture flow: identity, age, current club, parent contact, scouting notes. Submitting creates the `tt_prospects` row, completes the task, and chains to

`InviteToTestTrainingTemplate` assigned to the HoD.

- Default assignee: the user who initiates (the scout themselves).
- Form: `LogProspectForm` — two-screen, persistent drafts on. Submitting writes the prospect row and stores `prospect_id` in the task response payload so the chain step can read it.
- Default deadline: 14 days (just to surface stale-not-yet-completed tasks; the assignee is the initiator so this is generous).
- `chainSteps()` : one step `notify_hod_review` → `InviteToTestTrainingTemplate` with context inheriting `prospect_id` .
- Required cap: `tt_edit_prospects` (Scout, HoD, Admin).

A duplicate-detection check during submit: fuzzy-match against existing prospects and `tt_players` on first-name + last-name + age-group + current-club (Levenshtein-based, threshold 85%). Match found → "this might be a duplicate of {existing prospect}" prompt with link/proceed buttons. False positives are preferable to false negatives in this domain.

`InviteToTestTrainingTemplate` . Spawned automatically when `LogProspectTemplate` completes. Assigned to HoD (via the existing `RoleAssigneeResolver` for `head_of_development`). HoD opens the task, picks an existing or new `tt_test_trainings` session, composes the parent-facing invitation (email/SMS via the existing invitations-config infrastructure), submits.

- Form: `InviteToTestTrainingForm` — session picker, message composer with template, optional second-prospect picker if HoD wants to invite multiple to the same session.
- Default deadline: 7 days from spawn.
- `chainSteps()` : one step `await_parent_confirmation` → `ConfirmTestTrainingTemplate` with `prospect_id` and `test_training_id` in context.
- Required cap: `tt_invite_prospects` (HoD, Admin).

`ConfirmTestTrainingTemplate` . A "soft" task for parent confirmation. The task is technically assigned to the HoD (because parents aren't in the workflow user model the same way staff are), but its completion is triggered by an inbound parent action — clicking the "yes, we'll come" link in the invitation email, which hits a public endpoint that completes the task on the parent's behalf. If the parent doesn't confirm by deadline (default 5 days), the task goes overdue and the HoD sees it on their tasks dashboard.

- Form: a small admin form with three buttons: "Confirmed (parent agreed)", "Declined (parent withdrew)", "No response — mark no-show".
- Default deadline: 5 days, anchored to the test-training date itself (deadline = `test_training.date - 1 day`).
- `chainSteps()` :
  - If outcome = confirmed → `RecordTestTrainingOutcomeTemplate` .
  - If outcome = declined → terminal (archive prospect with reason `parent_withdrew`).
  - If outcome = no-show → terminal (archive with reason `no_show`).
- Required cap: `tt_invite_prospects` .

`RecordTestTrainingOutcomeTemplate` . Spawned the day after the test-training session. Assigned to the coach

who ran the session (`tt_test_trainings.coach_user_id`), with HoD cc'd. Coach records observation and recommendation; HoD reviews and decides at submit.

- Form: per-attendee row (multiple prospects can be submitted in one batch — the form supports the operational reality of "one HoD, one decision pass after one session"). Each row: coach observation field + recommendation dropdown (`admit_to_trial` / `decline` / `request_second_session`).
- Default deadline: 7 days from session date.
- `chainSteps()` :
  - For each attendee with `admit_to_trial` : dispatches a side-effect step that creates a `tt_trial_cases` row via the existing `TrialCasesRepository` (not a workflow task — the trial case is the next *entity*, not the next task). Sets `prospect.promoted_to_trial_case_id` . Spawns `ReviewTrialGroupMembershipTemplate` (the next task in the chain) assigned to HoD with deadline `now + 90 days` .
  - For each attendee with `decline` : triggers the configured decline-letter via existing `tt_trial_letters_generated` , archives the prospect with reason `declined` . Terminal.
  - For each attendee with `request_second_session` : spawns a fresh `InviteToTestTrainingTemplate` (loop back to the invitation stage). The HoD can re-invite to a later session.
- Required cap: `tt_decide_test_training_outcome` (HoD, Admin).

`ReviewTrialGroupMembershipTemplate` . A long-running review task that punctuates the trial-group experience. Spawned 90 days after admission, then re-spawned every 90 days while the trial case stays in `continue_in_trial_group` decision state. Assigned to HoD.

- Form: a snapshot of the player's trial-group performance — attendance %, evaluations averaged, coach observations — plus a decision dropdown: `offer_team_position` / `continue_in_trial_group` / `decline_final` .
- Default deadline: 14 days from spawn.
- `chainSteps()` :
  - `offer_team_position` → spawns `AwaitTeamOfferDecisionTemplate` (the parent + player decide).
  - `continue_in_trial_group` → after onComplete the trial case's `continued_until` is bumped, and the template re-spawns itself in 90 days.
  - `decline_final` → trial case decision = `deny_final` , prospect archived. Terminal.
- Required cap: `tt_decide_trial_outcome` (HoD, Admin).

The template-config admin (existing — `tt_workflow_template_config` table) lets clubs override the deadlines per-template per-club without code changes. A club that prefers shorter or longer review cycles flips the override.

Migration `0060_onboarding_workflow_templates` registers the five templates in `TemplateRegistry` at boot. No data migration; templates are code, not config.

REST + dispatcher. A new endpoint `POST /wp-json/talenttrack/v1/prospects/log` calls `TaskEngine::dispatch('log_prospect', $context)` to start the chain. Subsequent stages are handled entirely by `TaskEngine` chain spawning — no bespoke orchestration. The existing `tasks-dashboard` view automatically picks up the new templates because its row source enumerates from `TemplateRegistry::all()` .

3. `feat-trial-cases-rolling-membership`

Unchanged from yesterday's draft (kept here for sequencing clarity). Three new decision values, `continued_until` column, per-club trial-group pseudo-team for attendance integration.

The interaction with the workflow chain: when `RecordTestTrainingOutcomeTemplate` completes with `admit_to_trial`, the side-effect creates the trial case AND the `ReviewTrialGroupMembershipTemplate` task. The trial case and the task evolve in lock-step — the task's decision drives the trial case's `decision` and `continued_until` columns; the trial case's data drives the task's review form content. The two are tightly coupled by design; `tt_trial_cases.id` is stored in the task's context.

4. `feat-pipeline-widget` — `OnboardingPipelineWidget`

The visualisation. Implemented as a new widget in the persona-dashboard catalogue, registered alongside the existing `KpiCardWidget`, `DataTableWidget`, etc. The widget pivots existing workflow-task data and existing trial-case data — no new queries against bespoke pipeline tables, because the only new pipeline tables are identity (`tt_prospects`) and session metadata (`tt_test_trainings`).

Visual shape. A horizontal flow diagram, six stages from left to right:

```
┌──────────────┐  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐  ┌──────────
│  Prospects   │  │   Invited    │  │ Test Training│  │ Trial Group  │  │  Team Offer  │  │   Joined
│      12      │→ │      4       │→ │      3       │→ │      8       │→ │      2       │→ │      5
│ (2 stale)    │  │ (1 overdue)  │  │              │  │              │  │ (1 overdue)  │  │
└──────────────┘  └──────────────┘  └──────────────┘  └──────────────┘  └──────────────┘  └──────────
```

Each stage is one column showing: stage label, count of active items, count of stale items (overdue or > 30 days unchanged) in red. Clicking a column expands it inline (or below — see size variants) to show up to 5 cards: prospect name + age group + days-in-stage + assignee + a "Open task" link that navigates to the relevant workflow task or trial case.

Stage-to-data mapping. Each column queries:

- **Prospects column:** `tt_prospects` rows whose most recent workflow task is `LogProspectTemplate` (still in_progress) OR no task yet exists OR most-recent task is `LogProspectTemplate` completed but `InviteToTestTrainingTemplate` not yet open. (Captures both "scout still drafting" and "logged, awaiting HoD invite.")
- **Invited column:** prospects with an open `InviteToTestTrainingTemplate` or `ConfirmTestTrainingTemplate` task.
- **Test Training column:** prospects with an open `RecordTestTrainingOutcomeTemplate` task.
- **Trial Group column:** prospects with `promoted_to_trial_case_id IS NOT NULL` AND the trial case decision is null OR `continue_in_trial_group`.
- **Team Offer column:** prospects-as-players with an open `AwaitTeamOfferDecisionTemplate` or trial case `decision = offered_team_position` and `decision_made_at < 14 days ago`.
- **Joined column:** players whose `promoted_to_trial_case_id IS NOT NULL` AND `tt_players.status = 'active'` AND the joined-team transition happened in the last 90 days.

The mapping is a single SQL UNION-pivot per club: ~6 small queries, each cached for 60 seconds.

Size variants (following the existing widget convention):

- **`s` (small, 1×1 grid cell):** A single bar showing the six stage counts horizontally without expansion. Useful as a compact KPI replacement on a small persona dashboard.
- **`m` (medium, 2×1):** Stage counts plus per-stage stale-count badges, no card expansion. Good for the persona dashboard sidebar.
- **`l` (large, 3×2):** Stage counts plus stale badges plus inline-expand-on-click showing up to 5 cards per stage. The default for the HoD landing.
- **`xl` (full width, 4×3):** Same as large but always-expanded, all six columns showing cards simultaneously, plus a filter bar at the top (by scout, by age group, by date range).

The HoD landing template (per #0071) gets the widget added at `xl` in the slot directly below the team grid. The Scout's landing gets it at `m` showing only their own prospects (the matrix scope `RCD self` is honoured by the underlying queries). The Academy Admin's default template is a tile-driven control panel for system administration (Configuration, Authorization, Usage stats, Audit log, Migrations, etc.) and not an operational surface — putting an `xl` widget there would make assumptions about whether the Admin is also a daily pipeline operator. Instead, the Academy Admin landing gets a small KPI strip with three pipeline KPIs (`prospects_active_total` , `prospects_stale_count` , `prospects_promoted_this_season`) and a new navigation tile "Onboarding pipeline" pointing at the standalone view (see Section 6 below). Admins who *are* daily pipeline operators (small academies where one person wears multiple hats) can drag the widget onto their landing via the editor — but that's an opt-in, not the default.

Interactivity.

- Hover on a column title shows a tooltip: which workflow templates feed this stage.
- Click on a column expands the cards inline (in `l`) or scrolls them into a side panel (in `xl`).
- Click on a card navigates to the underlying task or trial case.
- The `xl` variant has filter chips at the top (the existing filter-chip component from the TasksDashboard view).
- Drag-and-drop is **not** implemented. Advancing a prospect to the next stage is "complete the current task" — that's done via the task's form, not a drag. The widget is read-only navigation; the workflow engine drives state changes.

This is a deliberate departure from yesterday's design. Drag-to-advance was the operationally riskiest affordance in the kanban draft (every drag is a state mutation; some drags would skip steps inappropriately). With the workflow engine driving transitions, "advance" means "complete the assigned task" — which goes through the task's form, the form's validation, and the chain's spawning logic. Operators advance by *doing the work*, not by dragging a card.

Performance. Server-rendered with placeholder counts and AJAX-loaded card content per-column — same pattern as existing KPI widgets. Server caches the per-club pivot for 60 seconds via `wp_cache_*` . For clubs with 100+ active prospects in the funnel the visible pixels are bounded.

Mobile (≤720px). Columns stack vertically; counts and stale badges stay; card expansion is via tap-to-toggle-accordion. The "+" button to log a new prospect is fixed bottom-right per the existing scout-mobile convention.

Caps and visibility. The widget renders if the user has `tt_view_prospects` . Displayed counts are scoped to what the user can see — a scout sees only their own prospects everywhere; HoD sees everything. The scope filter is applied at the SQL layer, not the rendering layer, so a scout literally cannot see the HoD's wider data.

5. KPI suite — ten new KPIs registered with `KpiDataSourceRegistry`

The persona dashboard's existing 25 KPIs get ten new pipeline-related KPIs that compose with `KpiCardWidget` and `KpiStripWidget`. Each is a small file (~30 lines) implementing `KpiDataSource`, registered in `CoreKpis::register()`.

The ten KPIs, with the relevant persona context, the value they compute, and *why each matters to that persona*:

#	KPI id	Label	Persona	Why it matters
1	`prospects_active_total`	Actieve prospects	ACADEMY (HoD, Admin)	The pipeline's headline number — total in any non-terminal stage. Anchors the funnel question.
2	`prospects_logged_this_month`	Prospects deze maand	ACADEMY	Discovery rhythm. A month with low scouting activity is its own warning signal.
3	`prospects_stale_count`	Vastlopende prospects	ACADEMY	Prospects with no task progress in > 30 days. The single most-actionable KPI on the dashboard — every stale prospect is either a missed opportunity or a GDPR liability.
4	`test_trainings_upcoming`	Komende proeftrainingen	ACADEMY	Sessions scheduled in the next 14 days. Operational planning anchor.
5	`trial_group_active_count`	Spelers in trialgroep	ACADEMY	Active trial-group membership. Drives the per-age-group capacity conversation ("we have 8 in U13 trial group, that's 2 over our usual cap").
6	`trial_decisions_pending`	Beslissingen in trialgroep	ACADEMY	Open `ReviewTrialGroupMembershipTemplate` tasks. The HoD's "you owe a decision" surface.
7	`team_offers_pending_response`	Aangeboden plekken open	ACADEMY	Players who have been offered a team position but haven't yet accepted/declined. Time-sensitive — every day of delay is a recruitment risk.
8	`prospects_promoted_this_season`	Doorstroming dit seizoen	ACADEMY	Prospects → academy players in the current season. The lagging-indicator for funnel effectiveness. Pairs with #2 to compute the funnel conversion rate.
9	`my_prospects_active`	Mijn actieve prospects	COACH (Scout-context)	Per-scout: prospects I logged that are still in any non-terminal stage. The scout's personal pipeline tile.
10	`my_prospects_promoted`	Mijn succesvolle	COACH	Per-scout: prospects I logged that promoted to academy team within the current or prior season. The scout's "I helped find these players" trophy KPI —

		scoutings		and a real coaching-development question for the HoD when reviewing scouts.

Three observations on the KPI selection:

- **Eight ACADEMY-context, two COACH-context (scout-flavoured).** Other personas don't get pipeline KPIs by default — Head Coach doesn't run scouts, Player and Parent shouldn't see prospect data at all. The KPI count distribution mirrors who actually uses pipeline data day-to-day.
- **The "stale count" KPI is the most operationally consequential.** It's the only one that triggers GDPR concerns (every stale prospect is a candidate for the auto-purge cron). Defaulting it onto the HoD landing is right; clubs can remove it via the editor if they want to look elsewhere. The stale threshold (default 30 days) is configurable per club via the existing lookup `prospect_stale_threshold_days`.
- **The "my prospects promoted" KPI is the lagging-indicator that closes the loop.** Scouts need to see the impact of their work, and HoDs need a number to anchor the "are our scouts effective" conversation. Without this KPI, the front of the funnel is invisible to the people doing it.

Each KPI file lives at `src/Modules/PersonaDashboard/Kpis/{Name}.php` following the existing `ActivePlayersTotal.php` pattern. Each registers in `CoreKpis::register()` with one line. Tests follow the existing convention: a unit test per KPI asserting the SQL returns expected values for a synthetic dataset.

Defaults in shipped templates. The HoD's default template (per #0071's enhanced HoD landing) gets four pipeline KPIs added to its existing strip: `prospects_active_total`, `prospects_stale_count`, `trial_decisions_pending`, `team_offers_pending_response`. The Scout's default template (which currently has no KPI strip) gets a small KPI strip with `my_prospects_active`, `my_prospects_promoted`, and the existing `recent_scout_reports`. The Academy Admin's default template is tile-driven (Configuration, Authorization, Usage stats, etc., on top of a `system_health_strip`) and does not have a KPI strip today. This child adds a small three-KPI strip above the existing tile grid: `prospects_active_total`, `prospects_stale_count`, `prospects_promoted_this_season`. These three are read-only awareness KPIs — an Admin scanning the landing sees "12 active prospects, 2 stale, 5 promoted this season" without needing to be a daily operator. If they want to act on it, the new "Onboarding pipeline" navigation tile (Section 6) is the next click.

Per-club overrides via the editor work as for any other KPI. Clubs that don't run scouting (rare but possible) can hide all ten via the editor; the data is still there, just not surfaced.

6. Standalone pipeline view — `?tt_view=onboarding-pipeline`

A thin standalone view that renders the `OnboardingPipelineWidget` at `xl` size on a single-column page. Three reasons to ship it:

- **Academy Admin.** Their default template is a control panel for system administration — Configuration, Authorization, Usage stats, Audit log, Migrations, Help — and it shouldn't be retrofitted into an operational page. They get a small KPI strip + a navigation tile pointing at the standalone view. The tile gives them awareness; the standalone view is where they go when something on the KPIs flags an action.
- **HoD focus mode.** A HoD whose landing already has the widget at `xl` may still want a full-screen view (no team grid, no other widgets, just the pipeline). The standalone view serves that.
- **Scout deep-dive.** Scouts whose landing has the widget at `m` (compact, just-a-tile) can click through to the standalone view to see their full pipeline at `xl` with filter chips active.

Implementation. New view file `src/Modules/Prospects/Frontend/FrontendOnboardingPipelineView.php` , ~30 lines: gate on `tt_view_prospects` , render a header, render `OnboardingPipelineWidget` at `xl` with no per-template config (default everything). The widget code is reused completely; the view is a wrapper. URL: `?tt_view=onboarding-pipeline` . Registered in `CoreSurfaceRegistration` like every other frontend view.

Cap and matrix: gated on `tt_view_prospects` . The widget already filters its data by the user's matrix grants — a scout sees only their own prospects on the standalone view exactly as in the dashboard widget. No new matrix entity; the standalone view inherits the `prospects` entity's grants.

Navigation tile. The view is added to the `CoreSurfaceRegistration` as a `tile` -eligible surface. The Academy Admin's default template gets the tile added between the existing `usage-stats` and `audit-log` tiles. The HoD landing gets it as a small "see full pipeline" link inside the widget's `xl` instance, not as a separate tile (the widget is the pipeline; a separate tile to the same content would be redundant). The Scout landing gets the tile added next to the existing scout tiles.

Mobile. Same responsive behaviour as the widget — columns stack vertically on mobile. The standalone view's only mobile-specific addition is a more prominent "+ New prospect" floating action button at the bottom-right, since the standalone view is the most likely entry point for scouts on phones.

This is genuinely small — about 30 lines of view code plus three navigation-tile registrations. The win is that anyone who wants the pipeline at full size has somewhere to go, and the Academy Admin's tile-driven landing stays a tile-driven landing.

Wizard plan

No new wizards in the traditional sense. The five `TaskTemplate` classes ARE the user-facing flows, and they reuse the existing Workflow module's task-form rendering rather than the persona dashboard's wizard chrome. This is a deliberate convergence: the workflow engine already has its own form-rendering pattern (per `FrontendTaskDetailView`), and using it instead of the wizard chrome means tasks behave consistently with the existing five workflow templates (post-game eval, self-eval, etc.).

The "+ New prospect" entry-point button is the only non-task-form UI element introduced. It calls `TaskEngine::dispatch('log_prospect', $context)` which creates the first task and immediately opens it for the scout. No wizard needed — the task form IS the form.

Existing trial-case wizards in the Trials module continue to work; the three new decision values land as new dropdown options.

Out of scope

- **Bulk prospect import from CSV.** Single-prospect logging is the v1 path; CSV import is a separate spec.
- **Public-facing "register interest" form.** Some clubs want a form on their website where parents submit kid's details. Reserved as feature flag `prospects_self_registration` for future. Out of scope because the abuse-prevention surface (rate limit, CAPTCHA, consent UX, GDPR for self-registered minors) needs its own design.
- **Native mobile app for scouts.** The existing `?tt_view=` infrastructure is responsive; the LogProspect form works on a phone browser. A dedicated app is a separate product question.
- **Multi-club shared scouting pools.** Two academies in the same region wanting "we've seen this kid" coordination. Real opportunity at the multi-tenant data-sharing layer; not v1.

- **Drag-to-advance on the pipeline widget.** Discussed and rejected. State changes flow through the workflow engine; advancing means completing a task. The widget is read-only navigation.
- **Standalone "pipeline" view at `?tt_view=onboarding-pipeline`** — see Section 6 below. The widget is the visualisation; the standalone view is a thin one-page wrapper around it for personas without a natural landing slot for an `xl` widget (notably the Academy Admin). Same rendering code, same data, different entry point. Roughly 30 lines of view-rendering wrapper plus a navigation tile.
- **Automated talent-detection.** Computer vision against match video. Future opportunity if the market matures; not v1.
- **Linking the existing scout-reports module to prospects.** Today scout reports are about academy players; making them about prospects expands the reports renderer. Out of scope; prospect notes live in `tt_prospects.scouting_notes`.
- **Per-stage SLA tracking and alerting.** The KPI for "stale prospects" surfaces them; an alerting layer (email at threshold, Slack notification, etc.) is a separate spec. Worth considering in v2 once clubs report on real funnel rhythms.

Acceptance criteria

`feat-prospects-entity`:

- `tt_prospects` and `tt_test_trainings` tables exist after migration `0059`. The prospect table has no `status` column.
- Two new matrix entities seeded with the documented persona scoping. Scout's "own prospects only" scope is enforced at the SQL layer, verified by a security test.
- `LegacyCapMapper` bridges the four new caps. `PlayerDataMap` includes both tables.
- `tt_prospects_retention_cron` runs daily and purges per the documented thresholds, with audit-log entries.

`feat-onboarding-task-templates`:

- Five new `TaskTemplate` classes are registered in `TemplateRegistry` after boot.
- Completing `LogProspectTemplate` automatically dispatches `InviteToTestTrainingTemplate` via the existing `ChainStep` mechanism.
- Completing `RecordTestTrainingOutcomeTemplate` with `admit_to_trial` creates a `tt_trial_cases` row AND dispatches `ReviewTrialGroupMembershipTemplate`. Both are atomic with task completion.
- `ConfirmTestTrainingTemplate`'s public confirmation endpoint accepts a parent's click and completes the task without requiring login.
- Duplicate detection at log-prospect time surfaces ≥85% confidence matches.
- The existing `tasks-dashboard` view picks up the new templates without modification (because it enumerates from `TemplateRegistry`).

`feat-trial-cases-rolling-membership`:

- Three new `DECISION_*` constants exist on `TrialCasesRepository`. Trial-decision wizard's dropdown surfaces all six options.

- `continued_until` column exists after migration `0060`.
- Reopening a decided case (`tt_trial_case_reopen` cap) transitions it from `decided` back to `open` with `previous_decision` populated.
- Trial-group pseudo-team is created per club + age group; trial-case players auto-added via `tt_team_people`.
- Trial-group attendance flows through `tt_attendance` and shows up in the HoD landing's team grid.
- Player status values `trial`, `offered`, `trial_declining_team` are documented and used.

`feat-pipeline-widget`:

- `OnboardingPipelineWidget` is registered in `WidgetRegistry` after `CoreWidgets::register()`.
- The widget renders six columns at `l` size with the documented stage-to-data mappings.
- Counts are scoped to the user's matrix grants — a scout sees only their own prospects.
- Card expansion shows up to 5 cards per column.
- Mobile (≤720px) stacks columns vertically; the "+" button opens the LogProspectTemplate task.
- Per-column data is server-rendered with AJAX progressive enhancement; cache TTL is 60 seconds.
- The widget is added by default to the HoD and Scout templates. The Academy Admin gets a 3-KPI strip + navigation tile to the standalone view, not the widget itself. Per-club overrides preserved.

Standalone pipeline view (Section 6):

- New view at `?tt_view=onboarding-pipeline` renders `OnboardingPipelineWidget` at `xl` size on a single-column page, gated on `tt_view_prospects`.
- The Academy Admin's default template includes the new "Onboarding pipeline" navigation tile linking to the standalone view.
- Scout-scope filtering is honoured on the standalone view (a scout sees only their own prospects).
- Mobile renders stacked columns + a floating "+ New prospect" action button bottom-right.

KPI suite:

- Ten new KPIs registered in `KpiDataSourceRegistry`.
- Each KPI's `compute()` method returns sensible values for the documented contexts.
- Eight are tagged `ACADEMY` context and two `COACH`. Player and Parent personas cannot select pipeline KPIs in the editor (the editor's persona-context filter excludes them).
- Default templates for HoD, Academy Admin, and Scout include the documented subset of pipeline KPIs.
- Each KPI has a Dutch translation in `talenttrack-nl_NL.po`.

Notes

Documentation updates.

- `docs/onboarding-pipeline.md` and Dutch mirror — new doc. Operator guide for the full pipeline using the workflow-task framing. Walks through the chain, explains why the visual surface is a widget rather than a

kanban, documents the GDPR retention rules.

- `docs/access-control.md` and Dutch mirror — note the two new matrix entities and the scout's "own prospects only" scope.
- `docs/persona-dashboard.md` — document the new widget and the ten new KPIs.
- `docs/modules.md` — extend the Workflow module entry with the five new templates.
- `docs/trials.md` — document the rolling-membership rework.
- `docs/gdpr-compliance.md` (from #0073) — add a section for prospects.
- `languages/talenttrack-nl_NL.po` — every user-facing string. Suggested NL: "Prospect" (loanword used in NL football), "Proeftraining", "Trialgroep", "Doorstroming", plus the ten KPI labels. Defer to native-speaker review.
- `SEQUENCE.md` — append `#0074-epic-onboarding-pipeline.md` with four child entries.
- `CHANGES.md` — entry per child describing the user-facing change.

`CLAUDE.md` updates.

- §3 (data model) — add the prospect tables to the "core domain entities" list. Note: "Prospect lifecycle is driven by workflow tasks, not by a status column on the prospect entity. The current stage is derived from the most recent task on the chain. This avoids dual-state-machine drift."
- §4 (Workflow module conventions) — add a paragraph: "The onboarding pipeline (#0074) is the first cross-module use of the Workflow engine. Future cross-module flows should follow the same pattern: small entity for identity, task templates for stages, ChainSteps for transitions, persona-dashboard widget for visualisation. Avoid bespoke state machines that parallel the engine."

Two design judgement calls worth surfacing again.

The widget-not-kanban decision. This is the most important architectural choice in this redraft. By making the pipeline a render of workflow data rather than a parallel state machine, the entire feature inherits the engine's existing capabilities (deadlines, overdue surfacing, audit trail, per-club overrides, the existing tasks-dashboard) for free. The cost: there is no separate "pipeline page", just the widget. For most clubs the widget is enough; for the rare club that wants a fully-immersive pipeline experience, the `xl` size variant is the answer. If feedback after pilot rollout is "we want a dedicated page," it's a one-day wrap of the existing widget — not a redesign.

The "no status on prospect" decision. Yesterday's draft put a status enum on `tt_prospects`. This redraft drops it because the workflow tasks ARE the status. The only state change on the prospect itself is `archived_at` (soft-delete on terminal outcomes). This eliminates the dual-source-of-truth problem that bites every pipeline implementation eventually. The downside is that querying "give me all prospects in stage X" requires a join through `tt_workflow_tasks` — the queries are slightly more complex. The net is positive: complexity moves from "two state machines I have to keep in sync" to "one state machine with a join." Documented in the spec and `CLAUDE.md` so the next person doesn't try to add a `status` column back.

Effort estimate.

- Entity: ~250 LOC (two tables, retention cron, matrix seeding)
- Templates: ~600 LOC (5 templates × ~80 LOC + forms ~150 LOC + duplicate detection ~50 LOC + tests ~150 LOC)

- Trial cases rework: ~400 LOC (decision values + rolling membership + pseudo-team + reopening)
- Widget + KPIs + standalone view: ~700 LOC (widget ~250 LOC including the pivot SQL + 10 KPIs × ~30 LOC + standalone view wrapper ~30 LOC + nav tile registrations + tests ~70 LOC)
- Docs: ~200 LOC

Total: ~1,200–1,400 LOC across four child PRs. About 50% smaller than the kanban draft. Largest single PR is the templates child. Most operationally consequential is the trial-cases rework (changes existing behaviour).

Recommended sequence for landing: entity → templates → widget+KPIs → trial-rework. Swapping the last two from the original ordering means the widget can ship against the existing trial decisions and gain the new ones in a follow-up patch — smaller PRs, the visual win lands sooner, and any pilot demo (e.g. a JG4IT-style review) gets the most demonstrable feature first.